Supplement

Gemma 4 Gives Its Failing Patches Full Marks

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Paper: `rubber-stamp.pdf` Kaggle paper track

This supplement accompanies *Gemma 4 Gives Its Failing Patches Full Marks* (`rubber-stamp.pdf`). Its sections follow the paper. Sections A and B give the setup of the paper's Section 2. Sections C to F give the detail behind Sections 3.1 to 3.4, and Section G gives the cost. Terms, patch sets and measures are those of the paper. Every number traces to a file in `scripts/derived/` of the repository, https://github.com/Vein05/gemma4-paper-track, and every test names its written plan in `spec/`. The raw runs behind them, including every agent and judge trace, are in the Hugging Face dataset https://huggingface.co/datasets/vein05/gemma4-paper-track-data. Analyses chosen after a result was seen are marked exploratory.

A Setup details and judge prompt

Issues. The held-out issues are the first 200 of a fixed random order (seed 0) of SWE-bench Verified issues outside the Mini issues for which at least one of the five other writers has a usable patch (447 issues). Five pilot issues drawn first were left out of the sample. A usable patch is non-empty after removing test files and at most 30,000 characters long.

Agent. The 4-bit competition checkpoint `gemma-4-31B-it-qat-w4a16-ct` (revision `52f3f65b`) ran in mini-SWE-agent v1.7.0 with its SWE-bench configuration unchanged (250 steps, temperature 0, 60-second command timeout), served by vLLM with a 131,072-token context on one NVIDIA L40S. Docker was replaced by Apptainer sandboxes of the same SWE-bench images. Grading used SWE-bench 4.1.0's grading code in the same kind of sandbox.

Judges. Outside judges for a model exclude its own family and, for each patch, the family of the patch's writer. Gemma as judge is the hosted model. OpenRouter routed its 1,064 held-out calls to nine providers. Most of them (983) went to providers that listed 4-bit or 8-bit weights on 2026-10-05. Of the rest, 79 went to providers that listed no precision and 2 to a provider that served bf16. We did not record the precision of each call (provider per call: `scripts/derived/fresh_gemma4-31b.jsonl`). In an earlier one-word check on 173 other patches, the hosted model pinned to a bf16 provider and the 4-bit checkpoint gave the same answer in 331 of 344 calls. The re-ask limit was 8,000 output tokens for our own patch sets and 3,000 for the mixed-harness set. Other writers' patches on the Mini issues keep their mixed-harness scores.

Gemma as judge, served locally. We repeated the held-out judging with the competition checkpoint itself, served by vLLM 0.30.0 on one NVIDIA A100 at temperature 0 with thinking off, under a plan fixed before any score (`spec/gemma-31b-local-judge.md`). It gave a score to all 1,063 prompts on the first ask (rows: `scripts/derived/fresh_gemma4-31b-local.jsonl`).

On the paper's patches, the local judge gives 100 to 54% of Gemma's failing patches and to 32% of other writers'. The hosted model gives 61% and 35%. On failing patches, the local judge's leniency difference is +13.4 [6.9, 20.0]. For the hosted model it is +16.6 [10.3, 23.1]. The paired difference, hosted minus local, is +3.2 [−0.5, 7.2]. We detect no difference between the two judges, but the interval allows the hosted model to be up to 7.2 points more lenient. On passing patches, the local judge's leniency difference is +5.0 [0.8, 9.0]. For the

hosted model it is +3.4 [−2.0, 8.0]. The local judge's AUROC is 0.658 on Gemma's patches and 0.762 on other writers'. For the hosted model they are 0.622 and 0.751.

Prompt. Every judge received the same system and user messages at temperature 0 (`scripts/selfjudge_matrix.py`, `scripts/selection_test.py`). Braces mark the parts filled in per patch.

```
System:  You are an expert Python maintainer.  You review a proposed fix for a reported issue.  You
cannot run code; judge from the material given.

User:  A user reported the issue below for the repository {owner/repo}.
<issue>{issue text}</issue>
A coding agent produced the patch below.
<patch>{patch without test files}</patch>
Question:  after this patch is applied, will the maintainers' hidden tests for this issue pass?  The
hidden tests were written for the upstream fix.  They check the behaviour the issue asks for, and tests
that passed before must still pass.  Explain briefly, then end your reply with a final line that is
exactly SCORE: <n>, where n is your probability from 0 to 100 that the hidden tests pass.
```

Two measures used only in this supplement. A judge's AUROC on a set of patches is the chance that it scores a random passing patch above a random failing one, with ties counted as half. The self-judging gap is how much a model's AUROC drops from other writers' patches to its own, minus the mean drop of the outside judges on the same patches. A positive gap means the model separates its own passing and failing patches worse than the outside judges do.

Re-asks. An answer without a parsable score was asked once more with a larger output limit (Table 1). Most had run out of the 3,000-token limit before the score line.

Table 1. Answers without a parsable score on our own patch sets (1,063 patches per judge), re-asked once with an 8,000-token limit.

Judge	Re-asked	Recovered	Left out
Gemma 4 31B	1	0	1
gpt-oss-20b	14	14	0
Qwen3.5-9B	74	73	1
GLM-4.7-Flash	52	42	10
Qwen3-Coder	0	0	0
gpt-oss-120b	5	0	5
GLM-4.5	1	1	0
Kimi-K2	129	117	12
Llama-4-Maverick	0	0	0

Sensitivity to the re-ask (exploratory). Leaving out patches whose answer needed a re-ask, the held-out self-judging gap is +0.030 [−0.024, 0.083] (all judges' re-asked patches; 763 patches, 143 of Gemma's), +0.055 [0.001, 0.106] (only Kimi-K2's) and +0.063 [0.012, 0.115] (only Qwen3.5-9B's). Without the re-asked patches, Gemma's own AUROC rises from 0.622 to 0.670, its leniency on failing patches is +11.1 [3.8, 18.2] and its rubber-stamp rate 56%. On per-judge percentile ranks instead of raw scores, the held-out leniency is +0.085 [0.048, 0.122] on failing patches and +0.024 [−0.007, 0.055] on passing patches.

B Agent runs and grading

Agent runs. The 200 held-out issues ran as two jobs of 100. In the first, 98 runs submitted a patch, one ran out of the 131,072-token context and one stopped after repeated model-call timeouts. Three patches were empty. In the second, one submitted patch was empty. On the Mini issues, 49 runs submitted and one stopped after repeated timeouts. Empty patches and runs that stopped count as unresolved, as in SWE-bench. One Mini issue (`django__django-11848`) failed to start its sandbox and was run again. The Mini analyses use the second run's patch.

Reference-patch check. We graded the official reference patch of every issue we ran (5 pilot, 200 held-out, 50 Mini). 248 of 255 resolve in our environment. The first pass labelled 16 matplotlib and xarray issues as timeouts. Logs showed that the container build, not the tests, exceeded a 1,200-second limit, and with a 3,600-second build limit all but three resolved. The 7 issues whose reference patch does not resolve were dropped from the held-out set (`astropy-7606`, `matplotlib-20826`, `-23314`, `-26113`, `-26208`, `requests-2931`, `sphinx-9711`). Gemma resolved none of them.

Agreement with official labels. Before using our grading, we graded 30 gpt-oss-120b patches with known official labels, stratified by repository. 26 of 30 matched. The four misses were environment failures (a network-dependent test, Ghostscript file access) and patches that apply only with fuzzy matching on today's rebuilt images. Other writers' patches therefore keep their official labels.

Resolve rates. Gemma resolved 56 and 42 of the two held-out batches of 100, 98 of the 193 kept issues in all, and 22 of the 50 Mini issues. On the Mini issues two sphinx patches ran past the 1,800-second test limit. Counting them as failing, as SWE-bench does, changes the matched-harness gap from +0.066 to +0.081 [−0.010, 0.178].

Leftover files. Gemma's held-out patches often add a file it left behind, such as a reproduction script or a test project, and every judge sees that file in the diff. We removed every added file from all 957 held-out patches and had all nine judges score them again with the same prompt and settings, under a plan fixed before the scores (`spec/stripped-files-rescore.md`, `derived/stripped_files_rescore_2026-11-03.json`). On the 875 patches every judge scored, Gemma's leniency difference on failing patches is +16.1 [8.9, 23.2] without the added files and +18.3 [10.8, 25.1] with them. The paired difference is +2.2 [−4.6, 8.7]. On unchanged patches, a judge's score moves by 6 to 16 points between two runs, so this shift is within run-to-run noise.

C Rubber-stamping on three data sets

This section backs Section 3.1 of the paper. Gemma gives 100 to about half or more of its failing patches on the held-out issues, on SWE-bench Lite and on LiveCodeBench, and the outside judges do not share this trust. Five large models do not show it on their own patches. Several small open models do.

Held-out issues, per judge. Table 2 gives each judge's share of failing patches scored 100. Gemma gives 100 to 52 of its 85 failing patches (61%) and to 136 of the 393 failing patches by other writers (35%). No outside judge comes close to the 61% on Gemma's failing patches.

Table 2. Held-out issues, per judge: share of failing patches scored 100 on Gemma's patches and on the other writers' patches. AUROC on Gemma's patches. For outside judges, the drop from their AUROC on other writers' patches to their AUROC on Gemma's (the mean drop, 0.055, enters the self-judging gap).

Judge	Gemma's failing at 100	Others' failing at 100	AUROC, Gemma's	Drop
Gemma 4 31B	61.2%	34.6%	0.622	
Llama-4-Maverick	45.9%	38.7%	0.611	0.059
GLM-4.7-Flash	36.5%	24.9%	0.636	0.091
Qwen3.5-9B	21.2%	23.4%	0.685	0.050
GLM-4.5	10.6%	9.2%	0.778	0.053
Kimi-K2	1.2%	1.8%	0.766	0.055
gpt-oss-20b	0%	0%	0.665	0.038
gpt-oss-120b	0%	0%	0.656	0.074
Qwen3-Coder	0%	0%	0.717	0.022

Mini issues. On the 50 Mini issues, Gemma's patches come from four public harnesses (mixed-harness patches). Gemma gives 100 to 61% of its failing patches there and to 20% of other writers' failing patches. Its leniency difference is +20.9 [11.5, 30.2] (Table 6).

Two more data sets

We ran the paper's measures on two data sets outside SWE-bench Verified. Each had a written plan before any judge call (`spec/third-party-lite.md`, `spec/livecodebench-judging.md`). The rule in both plans was the same. The result holds if Gemma gives 100 to a larger share of its own failing items than of other writers' failing items, and the leniency difference on failing items has a 95% interval above zero. Both results hold (Table 3).

SWE-bench Lite, another team's run. Gemma 4 31B's patches for the 300 SWE-bench Lite issues come from a run that another team published on Hugging Face (`dougalldeepmind/2026-10-10-swebench-gemma4-lite-gemma-v2`, revision `1c5f3d3b`). We had no part in that run. It used the full-precision model with thinking on, sampled at temperature 1.0 (top-p 0.95), and ran mini-SWE-agent 2.2.1 with one attempt per issue. That team graded the patches, and 166 of 300 resolved their issue. We used their labels. The other writers are four entries of the SWE-bench Lite leaderboard (`SWE-bench/experiments`): DeepSeek-V3 in Moatless, SWE-Fixer with Qwen2.5-72B, entroPO with Qwen3-Coder-30B, and Claude 4 Sonnet in SWE-agent. We prepared all patches as in the paper. Of the 300 issues, 37 are also held-out issues and 7 are Mini issues, with different patches. We kept them.

LiveCodeBench, no agent. We generated Gemma's programs ourselves with vLLM, from the 4-bit competition checkpoint, at temperature 0 with thinking off, using LiveCodeBench's own prompt (release_v6, 1,055 problems; `spec/livecodebench-pilot.md`). Answers cut off at the default 2,000-token limit got a second try at 8,000 tokens. Gemma then passes 873 of the 1,055 problems (82.7%). We judged all 146 problems where Gemma's program ran and failed, and 146 problems where it passed, drawn within difficulty. The other writers are DeepSeek-V3 and Claude 3.5 Sonnet, with the programs they released on LiveCodeBench. We removed comments and docstrings from every program, because Gemma's long docstrings would tell a judge who wrote it. The prompt is the paper's, with a contest problem in place of the issue, a program in place of the patch, and a competitive programmer in place of the Python maintainer in the system prompt.

Judges. Our remaining OpenRouter credit did not cover nine judges. Before any score was read, we left out GLM-4.5 and Kimi-K2 on SWE-bench Lite, which leaves seven judges. On LiveCodeBench we also left out Qwen3-Coder as a judge and GPT-4o as a writer, which leaves six judges. Of 1,478 Lite patches, 1,455 were scored by all seven judges. Of 839 programs, 761 were scored by all six. Intervals resample issues or problems.

Table 3. The paper's measures on the two data sets. Rates of 100 are on failing items. "Outside" is the mean rate of the outside judges on Gemma's own failing items. Leniency is own minus other writers' items, in points. `derived/third_party_lite_2026-11-01.json`, `derived/lcb_judging_2026-11-01.json` and their `reasked-left-out` versions.

	SWE-bench Lite	LiveCodeBench
Gemma's own items: failing / passing	130 / 163	124 / 126
Other writers' items: failing / passing	695 / 467	403 / 108
Gemma gives 100: own / other writers'	63% / 40%	50% / 8%
Outside judges give 100 to Gemma's failing items	21%	15%
Leniency, failing items	+17.8 [12.1, 23.5]	+23.0 [16.6, 29.9]
Leniency, passing items	+2.5 [−1.2, 6.2]	+5.1 [−2.0, 12.2]
Self-judging gap	+0.078 [0.031, 0.123]	+0.061 [0.012, 0.114]
Without re-asked items: leniency, failing	+18.8 [12.5, 24.8]	+26.5 [18.3, 35.0]
Without re-asked items: self-judging gap	+0.080 [0.030, 0.128]	+0.097 [0.039, 0.160]

Unlike the held-out gap, neither new gap depends on the re-ask (exploratory). On LiveCodeBench, the outside judges also rank Gemma's programs much worse than other writers' programs (a drop of 0.174 in AUROC), so every judge shares most of Gemma's own drop there.

Limits of the LiveCodeBench data. A Claude model read and ran 20 of Gemma's failing programs (`data/livecodebench-pilot/hand-check/`). It found 14 real bugs. Three were grading errors: two float outputs within the stated tolerance and one valid answer among several. Three were correct algorithms that run too slowly in Python.

All LiveCodeBench problems predate 2025-04-07. DeepSeek-V3's pass rate falls from 77% on problems from the first half of 2023 to 16% on the first half of 2025. Gemma's stays at 86 to 99% in every half-year (answers cut off at 2,000 tokens left out), so we cannot rule out that Gemma saw the problems in training. The two other writers pass far fewer of the judged problems than Gemma (18% and 24%, against 50%). Gemma's failing programs are mostly on hard problems, and other writers' failing programs include problems Gemma solved. On the 146 problems Gemma failed, its leniency is still +20.3 [13.5, 27.1], and it gives 100 to 50% of its own failing programs against 12% of the other writers' (exploratory, `scripts/lcb_matched_check.py`).

Per-writer leniency (exploratory). We chose this analysis on 2026-11-01, after the results above (`scripts/per_writer_leniency.py`, `derived/per_writer_leniency_2026-11-01.json`). Table 4 splits "other writers" by writer. Gemma's own failing items get the most leniency in each data set. One other writer comes close in each SWE-bench set. On the held-out issues, Gemma gives 100 to 61% of Kimi-K2's failing patches, and its leniency on its own is only 5.6 [−5.1, 16.3] points higher than on Kimi-K2's. On SWE-bench Lite, the gap to Claude 4 Sonnet is 1.7 [−5.3, 8.7] points. In both SWE-bench sets, Gemma is lenient toward writers that pass more than half their issues and harsh toward writers that pass under a third. Qwen3-Coder, which passes 70%, gets neither.

Table 4. Gemma as judge, per writer, on failing items: its leniency against the outside judges (points) and the share it scores 100. Pass rate is over the writer's judged items.

Data set	Writer	Pass rate	Leniency, failing	Scored 100, failing
Held-out	Gemma 4 31B (own)	53%	+14.1	61% (52 of 85)
	Kimi-K2	61%	+8.4	61% (33 of 54)
	GLM-4.5	62%	+4.1	56% (34 of 61)
	Qwen3-Coder	70%	+0.4	48% (22 of 46)
	Llama-4-Maverick	27%	−6.9	16% (20 of 123)
	gpt-oss-120b	30%	−8.1	25% (27 of 109)
SWE-bench Lite	Gemma 4 31B (own)	56%	+12.9	63% (82 of 130)
	Claude 4 Sonnet	57%	+11.2	68% (86 of 126)
	entroPO, Qwen3-Coder-30B	46%	−3.2	36% (57 of 158)
	SWE-Fixer, Qwen2.5-72B	25%	−6.1	35% (77 of 219)
	DeepSeek-V3	32%	−15.5	32% (61 of 192)
LiveCodeBench	Gemma 4 31B (own)	50%	+10.5	50% (62 of 124)
	Claude 3.5 Sonnet	18%	−10.6	5% (11 of 225)
	DeepSeek-V3	24%	−15.0	12% (21 of 178)

Placebo check (exploratory). Some outside judges also give Gemma's failing items 100 more often than other writers'. On SWE-bench Lite, Llama-4-Maverick gives 100 to 65% of Gemma's failing patches and 43% of other writers'. We tested whether such judges are also lenient on Gemma's items. We treated each outside judge as if it had written them and computed its leniency the same way, against the other judges except Gemma. Maverick's value is −5.4 [−10.1, −0.7]. It gives high scores to many patches, but not more to Gemma's than the other judges do. On SWE-bench Lite and LiveCodeBench, Gemma's leniency exceeds the largest of the outside judges' by 12.8 [4.3, 16.8] and 14.2 [3.3, 22.2] points. On the held-out issues, Kimi-K2 as judge is also lenient on Gemma's failing patches, at +10.0 [5.0, 14.9] against Gemma's +16.6. There Gemma exceeds the largest outside judge by 6.7 [−2.7, 12.2] points.

Five large models judging their own patches

On the Mini issues, five of the other writers judged their own mini-SWE-agent patches, so the harness is matched for them by construction. None shows a gap (Table 5). The pooled gap over the five is −0.036 [−0.089, 0.016].

We also measured their leniency on their own failing patches (Table 6). The intervals come from the same issue resamples as the gap. We computed them on 2026-10-22, after the point estimates had been seen, under a plan that counts a model as lenient only if its interval stays above zero after a Bonferroni correction for five models. Each model wrote only 14 to 24 failing patches, so the intervals are wide. The pooled leniency difference over the five is +5.1 [−0.8, 11.0] points. Gemma's is +20.9 [11.5, 30.2] on the same issues. That is 15.8 [4.0, 27.7] points above the pooled five. By the planned rule, none of the five is shown to be lenient.

Llama-4-Maverick comes closest. Its leniency difference is +17.5 [0.6, 32.9], and its Bonferroni interval [−3.5, 38.4] includes zero. Maverick gives high scores to most failing patches, whoever wrote them. It scores 95 or above on 69% of other writers' failing patches and on 57% of its own. So it does not single out its own patches the way Gemma does.

On the held-out issues, all nine judges also scored the five writers' own patches, which all come from

mini-SWE-agent (Table 7). Each writer has 46 to 123 failing patches there, three to six times as many as on the Mini issues. We chose this analysis on 2026-10-22, after the Mini results above and a review of the paper (`spec/other-models-heldout.md`). The code is the Mini code unchanged, and it reproduces Gemma's held-out gap and leniency. The pooled gap over the five is −0.001 [−0.031, 0.029], and the pooled leniency difference is +0.4 [−2.5, 3.4] points. Gemma exceeds the pooled five by 0.075 [0.015, 0.133] in gap and by 16.3 [8.9, 23.4] points in leniency. By the same Bonferroni rule, none of the five is shown to have a gap or leniency. Llama-4-Maverick's leniency difference falls to +2.8 [−5.0, 10.8], and it gives 100 to fewer of its own failing patches than of other writers' (33% against 42%).

Table 5. Models judging their own patches on the Mini issues (all patches from mini-SWE-agent).

Model	AUROC, own patches	AUROC, other families' patches	Self-judging gap [95%]
Qwen3-Coder	0.691	0.786	−0.032 [−0.241, 0.189]
gpt-oss-120b	0.860	0.701	−0.046 [−0.133, 0.049]
GLM-4.5	0.757	0.752	−0.047 [−0.174, 0.116]
Kimi-K2	0.881	0.805	−0.036 [−0.113, 0.036]
Llama-4-Maverick	0.786	0.657	−0.019 [−0.133, 0.101]

Table 6. Leniency difference of models judging their own patches on the Mini issues, in points: mean leniency on own patches minus mean leniency on other writers' patches. Gemma's patches come from four harnesses, the others' from mini-SWE-agent.

	Own	Failing patches		Passing patches	Failing scored
Model	failing	[95%]	[Bonferroni 99%]	[95%]	100: own / others'
Qwen3-Coder	15	+4.1 [−6.1, 14.6]	[−8.2, 18.3]	+3.5 [−15.5, 20.8]	7% / 0%
gpt-oss-120b	24	+2.1 [−9.1, 13.0]	[−11.3, 15.6]	+12.1 [5.1, 19.9]	0% / 0%
GLM-4.5	14	+7.1 [−10.5, 26.3]	[−17.7, 31.6]	+4.4 [−11.4, 17.5]	7% / 16%
Kimi-K2	14	−5.4 [−17.6, 7.2]	[−22.1, 11.6]	+4.9 [−2.7, 12.4]	0% / 2%
Llama-4-Maverick	21	+17.5 [0.6, 32.9]	[−3.5, 38.4]	+7.9 [−3.7, 19.6]	43% / 40%
Pooled, five models		+5.1 [−0.8, 11.0]			
Gemma 4 31B	80	+20.9 [11.5, 30.2]	[8.4, 32.6]	+0.2 [−7.6, 8.7]	61% / 20%

Table 7. Models judging their own patches on the held-out issues (all patches from mini-SWE-agent; exploratory). Leniency is the difference on failing patches, in points.

Model	Own failing	Self-judging gap [95%]	Leniency, failing patches [95%]	[Bonferroni 99%]	Failing scored 100: own / others'
Qwen3-Coder	46	−0.040 [−0.100, 0.023]	−8.1 [−15.4, −0.5]	[−17.3, 1.9]	0% / 0%
gpt-oss-120b	109	+0.001 [−0.066, 0.073]	+2.9 [−3.3, 8.8]	[−5.0, 10.8]	0% / 0%
GLM-4.5	61	−0.048 [−0.113, 0.019]	+0.8 [−8.8, 9.7]	[−11.8, 12.3]	12% / 9%
Kimi-K2	54	+0.035 [−0.035, 0.103]	+3.5 [−3.9, 10.6]	[−6.2, 13.2]	4% / 1%
Llama-4-Maverick	123	+0.048 [−0.021, 0.117]	+2.8 [−5.0, 10.8]	[−7.7, 12.5]	33% / 42%
Pooled, five models		−0.001 [−0.031, 0.029]	+0.4 [−2.5, 3.4]		
Gemma 4 31B	85	+0.074 [0.022, 0.126]	+16.6 [10.3, 23.1]	[7.6, 24.5]	61% / 35%

Small open models on LiveCodeBench

We also asked whether smaller open models trust their own failing programs, under a plan fixed before any of them wrote a program (`spec/lcb-model-sweep.md`, exploratory). Each model ran on one NVIDIA A100 with vLLM at temperature 0, with thinking off where it has a switch. It wrote one program for each of the 292 judged LiveCodeBench problems with LiveCodeBench's own prompt, and LiveCodeBench's evaluator graded it. The model then scored its own programs and the 839 existing programs with the paper's LiveCodeBench prompt. These models have no shared outside panel, so Qwen3-32B (thinking off) scored every program as the reference judge. Leniency here is the model's score minus Qwen3-32B's score on the same program, own failing programs minus other writers' failing programs. Other writers leave out the model's family, so the Gemma models compare against Claude 3.5 Sonnet and DeepSeek-V3 only.

Table 8 gives the result. Three of the four usable models outside the Gemma family are lenient on their own failing programs: Nemotron-Nano-9B-v2, Seed-OSS-36B and OLMo-3.1-32B. gpt-oss-20b is not. Gemma 4 12B and 26B-A4B are lenient too, and Gemma 4 31B is the most lenient of all. By the readings fixed in the plan, own leniency is common among these models, and it is a trait of the Gemma family.

Two models are not usable. GLM-4-9B passes 1 of 292 problems, and only 121 of its 839 scores on existing programs parse. Phi-4-mini passes 8 problems. Its scores of its own failing programs are no higher than of other writers' failing programs, but Qwen3-32B scores Phi-4-mini's programs much lower than other writers' failing programs, and that difference appears as leniency. Mistral-Small-3.2-24B did not run, because vLLM could not load it in the shared server environment.

Table 8. Small open models on 292 LiveCodeBench problems, judging their own and other writers' programs. Leniency is own failing minus other writers' failing programs, with Qwen3-32B as the reference judge, in points with 95% intervals from 2,000 problem resamples. Gemma 4 31B uses its existing programs and scores. `derived/lcb_sweep_2026-11-04.json`.

Model	Pass rate	Own failing	Scored 100: own / others' failing	Leniency [95%]
Gemma 4 31B		122		+40.3 [31.8, 48.9]
Gemma 4 26B-A4B	57.2%	73	45% / 8%	+28.3 [15.8, 40.4]
Gemma 4 12B	41.4%	83	45% / 8%	+26.0 [15.7, 36.5]
Nemotron-Nano-9B-v2	32.5%	40	58% / 14%	+18.6 [6.5, 30.8]
Seed-OSS-36B	15.1%	243	69% / 53%	+13.4 [6.5, 19.6]
OLMo-3.1-32B	18.2%	134	30% / 9%	+12.1 [4.6, 19.8]
gpt-oss-20b	36.3%	171	2% / 1%	+4.2 [−2.8, 11.4]
GLM-4-9B (not usable)	0.3%	55	0% / 3%	
Phi-4-mini (not usable)	2.7%	280	3% / 1%	

D Where the extra trust goes

This section backs Section 3.2 of the paper. Gemma could trust its patches because it recognizes them as its own, or because it believes in the edit they make. The tests below point to the edit. None of them varies the writer with the patch held fixed, so none rules out recognition.

Gemma is not lenient in general. On other writers' failing held-out patches, Gemma's mean leniency is −2.6 points, so it scores them slightly below the outside judges do (`derived/gemma_sizes_2026-10-28.json`). Its extra trust is in its own failing patches only.

Patches the outside judges score the same (exploratory). Gemma's failing patches do look a little better to everyone. The outside judges give them a mean score of 57.2 on the held-out issues, against 46.8 for other writers' failing patches. We therefore compared Gemma's own failing patches only with other writers' failing patches that the outside judges score the same. We cut all failing patches into five bins of equal size by the outside judges' mean score and weighted each bin by the number of Gemma's own failing patches in it (`scripts/strong_writer_leniency.py`, `derived/strong_writer_leniency_2026-11-04.txt`). Gemma's leniency difference is then +13.6 [7.1, 20.3] on the held-out issues and +17.2 [11.3, 23.0] on SWE-bench Lite, against +16.6 and +17.8 without the match.

Other writers' failing patches that make Gemma's edit

Test. We fixed this test in a written plan before computing it (`spec/belief-vs-recognition.md`, exploratory). For each issue, we compared other writers' failing patches with Gemma's failing patch on the same issue. A patch's edit is the set of lines it removes or adds in files that already exist, with whitespace stripped. A patch makes Gemma's edit when the Jaccard similarity of its changed lines with Gemma's is at least 0.5. Two measures use Gemma's excess score on each failing patch, its score minus the outside judges' mean. The first is the extra trust in other writers' patches that make Gemma's edit over those that do not. The second is the extra trust in Gemma's own failing patch over other writers' patches that make its edit, on issues that have both.

Reading fixed in the plan. The edit explanation is supported on a data set if the first measure has an interval above zero. The second measure must also be small: its interval includes zero, or its estimate is at most half the paper's leniency on the same issues. Recognition is supported if the second measure's interval is above zero and its estimate is more than half that leniency. With fewer than 20 same-edit patches, or any other pattern, the result is inconclusive. The plan allowed the paper to state the edit explanation only if both data sets support it.

Result. On SWE-bench Lite, Gemma gives other writers' failing patches that make its edit 20.0 points more extra trust than the others, and the edit explanation is supported (Table 9). On the held-out issues the difference is 8.0 points, its interval reaches zero, and the result is inconclusive. On neither set do we detect extra trust in Gemma's own patch beyond the same-edit patches. These are small comparisons: 16 held-out issues and 25 Lite issues have both kinds of patch, and on the same 16 held-out issues Gemma's leniency on its own failing patches is itself only +0.6. By the plan's rule, the edit explanation is supported on one data set of two. With a looser cut of 0.3, the first measure is +6.2 [−0.7, 12.5] on the held-out issues and +17.4 [11.8, 23.1] on Lite, and the second is +1.2 and −1.1, both intervals including zero.

Table 9. Gemma's extra trust in failing patches by edit, in points with 95% intervals from 2,000 issue resamples. Same edit: Jaccard similarity of changed lines with Gemma's failing patch on the issue at least 0.5. `derived/belief_vs_recognition_2026-11-03.json`, cut 0.3 in `..._cut03_2026-11-03.json`.

	Held-out	SWE-bench Lite
Other writers' failing patches: same edit / other edit	36 / 322	45 / 630
Same edit minus other edit	+8.0 [−0.9, 15.9]	+20.0 [14.1, 25.9]
Issues with both Gemma's failing patch and a same-edit patch	16	25
Gemma's own failing patch minus same-edit patches	−5.1 [−17.7, 3.9]	−7.6 [−19.1, 2.4]
Paper's leniency difference on those issues	+0.6 [−11.5, 11.7]	+1.2 [−10.4, 12.0]
Reading by the plan	inconclusive	edit explanation

Gemma 4 12B and 26B-A4B reading the 31B's patches

Test. Two smaller Gemma 4 models scored the same held-out patches, under a plan frozen before any score (`spec/gemma-sizes-judges.md`). They come from the same family and read the same style of code, but they did not write the patches. Gemma 4 12B ran as `gemma-4-12B-it-qat-w4a16-ct`, the 4-bit format of the 31B checkpoint. Gemma 4 26B-A4B ran from its bf16 weights with vLLM's fp8 quantization, because Google publishes no 4-bit checkpoint of it. Both ran with vLLM 0.30.0 on one NVIDIA A100 at temperature 0 with thinking off, with the paper's prompt and parser and one re-ask. The 12B left 3 passing patches by other writers unparsed, so the analysis uses 954 patches on 193 issues. The 31B's numbers on failing patches do not change on this set.

Reading fixed in the plan. A size shares the extra trust if its leniency difference on the 31B's failing patches has an interval above zero. It does not share it if the interval includes zero and its upper end is below +8.3, half of the 31B's +16.6. Anything else is inconclusive. Separately, the extra trust is weaker in a size if the 31B's leniency difference minus the size's has an interval above zero.

Result. Neither size shows measurable extra trust in the 31B's failing patches (Table 10). By the plan, the 12B does not share it and the 26B-A4B is inconclusive, since its interval reaches +11.8. In both, the extra trust is weaker than in the 31B, by 16.2 and 12.7 points. Both sizes are valid judges by the plan's rule. At least 95% of their answers parse, none carries thinking tokens, and their AUROC on other writers' patches is 0.72 and 0.73, above the plan's 0.60.

Table 10. Three Gemma 4 sizes judging the 31B's held-out patches (954 patches). Leniency in points with 95% intervals from 2,000 issue resamples. `derived/gemma_sizes_2026-10-28.json`.

Judge	Gemma 4 31B	Gemma 4 12B	Gemma 4 26B-A4B
The 31B's failing patches scored 100	52 of 85	27 of 85	34 of 85
Other writers' failing patches scored 100	136 of 393	95 of 393	130 of 393
Leniency, other writers' failing patches	−2.6	−10.0	+0.3
Leniency difference, failing patches	+16.6 [10.3, 23.1]	+0.4 [−6.8, 7.4]	+3.9 [−4.1, 11.8]
The 31B's difference minus this size's		+16.2 [6.2, 26.1]	+12.7 [5.1, 20.8]
AUROC: the 31B's patches / other writers'	0.622 / 0.750	0.665 / 0.722	0.663 / 0.729

Own wrong program against other wrong programs on LiveCodeBench

Test. The small-model data of Section C allow two more measures per judge, fixed in a plan before they were computed (`spec/lcb-sweep-blind-spot.md`, exploratory, no new model calls). Both use the judge's excess score over Qwen3-32B. The first asks whether being wrong on a problem makes a judge more lenient toward other writers' wrong programs on that problem. It is the judge's excess on other writers' failing programs on problems it failed minus on problems it passed. Problems a judge fails are harder, so we subtract the same split computed with the other judges' pass and fail. The second compares, on problems where the judge and other writers both failed, the judge's excess on its own wrong program with its mean excess on the other writers' wrong programs.

Result. Being wrong on a problem does not make six of the seven judges more lenient toward other writers' wrong programs (Table 11). Nemotron-Nano-9B-v2 is the exception. We had predicted this effect for Gemma 4 31B, and it does not appear (+2.8 [−4.0, 9.4]). The extra trust is in the judge's own wrong program. Gemma 4 31B trusts its own wrong program 39.0 points more than other wrong programs for the same problem. The 12B, the 26B-A4B, OLMo-3.1-32B and Seed-OSS-36B do the same, with intervals above zero. Nemotron-Nano-9B-v2's estimate is positive, but its interval includes zero, and gpt-oss-20b shows no such trust.

Table 11. LiveCodeBench, seven judges, excess score over Qwen3-32B in points with 95% intervals from 2,000 problem resamples. `derived/lcb_blind_spot_2026-11-03.json`.

Judge	Being wrong, other writers' wrong programs	Own wrong program minus others' (problems)
Gemma 4 31B	+2.8 [−4.0, 9.4]	+39.0 [30.7, 47.7] (136)
Gemma 4 12B	+2.0 [−4.5, 8.5]	+28.4 [18.3, 38.7] (80)
Gemma 4 26B-A4B	−1.6 [−5.9, 2.4]	+23.6 [11.0, 36.6] (71)
Seed-OSS-36B	−1.8 [−16.1, 12.1]	+12.4 [5.1, 19.1] (232)
OLMo-3.1-32B	−0.2 [−10.1, 9.6]	+11.8 [3.6, 20.5] (126)
Nemotron-Nano-9B-v2	+6.9 [0.3, 13.3]	+13.5 [−3.3, 29.5] (38)
gpt-oss-20b	−2.2 [−10.0, 5.4]	+2.5 [−4.3, 9.9] (167)

What Gemma writes when it rubber-stamps

Gemma's reasons (exploratory). Gemma scored 52 of its own failing held-out patches 100. At least two outside judges scored 31 of these at 50 or less. We set this rule after a first look at the answers. For each case, a coder found the main defect, the reason the low-scoring outside judges give for their low score, and asked whether Gemma's answer names it (`data/hand-codes/`, with the codebook). Two coders coded every case on their

own. Both were Claude models: Sonnet, then Opus without access to the first coder's codes. The first coded 5 cases as naming the defect and the second 6. They agree on 26 of 31 (Cohen's $\kappa = 0.45$): both find the defect never mentioned in 23 and named in 3. Two of the five split cases come from choosing the main defect when the outside judges name several. The other three turn on whether Gemma describing the flagged change, with no doubt about it, counts as naming the defect.

The codebook also asked whether Gemma states the opposite of the outside judges' claim. The two coders agree on this in only 13 of 31 cases ($\kappa = 0.03$), so we do not report it. One split case is django-15037. There Gemma calls `to_fields=[target_field]` "the correct way to specify a non-PK reference", and GLM-4.5 (score 15) answers that Django's keyword is `to_field`. Two other low-scoring judges give a leftover script in the patch as their reason, and Gemma mentions that script. `derived/rubber_stamp_reasons_2026-10-22.json`.

Gemma reads the defective line and calls it correct (exploratory). Two more coders, Claude Sonnet models working apart and blind to the scores, sorted the same 31 cases by defect type and marked the defective lines. They agree with $\kappa = 0.71$, and `data/hand-codes/defect-type.csv` gives the five settled disagreements with their reasons. Twenty are wrong code, seven are a missing change and four are unclear. In 18 of the 20 wrong-code cases the defect sits on one or two marked lines. Two further coders then read Gemma's answer for each case and asked whether it endorses the marked line (`data/hand-codes/claims-coder-a.csv`, `claims-coder-b.csv`). Both coders find that it does in all 18. Gemma quotes or describes the defective line and calls it correct.

Two simple explanations that do not fit

Both checks use the mixed-harness Mini patches, where Gemma has several attempts per issue.

Patch size. Gemma's patches are much smaller than the other writers' (7.7 against 72.7 added lines on average). Among single-file failing patches, matched by size in four bins, Gemma's leniency on its own patches stays 24.1 points [12.7, 39.9] above its leniency on other writers' patches (Table 12).

Table 12. Gemma's leniency on single-file failing patches by number of changed lines, mixed-harness Mini patches. The size-matched difference over these bins (own minus others') is +24.1 [12.7, 39.9].

Changed lines	Own failing (n)	Others' failing (n)	Leniency, own	Leniency, others'
1–5	29	19	+14.2	−7.1
6–15	32	13	+11.2	−17.1
16–40	11	3	+2.6	−11.6
41 or more	5	5	+29.1	−5.6

Issues it cannot solve. Gemma scores 66% of its failing patches at 95 or above on issues that one of its attempts solved (29 patches) and 67% on issues none solved (51). On other writers' failing patches for the unsolved issues, it is slightly stricter than outside judges (−5.8 points). So "a model cannot verify what it cannot solve" does not fit.

E Checks that Gemma runs itself

This section backs Section 3.3 of the paper. If the trust is a belief about the fix, a check that Gemma runs should inherit it. We tested four such checks: thinking before scoring, the reproduction scripts Gemma writes while fixing an issue, a shell in which Gemma must test the patch, and LoRA fine-tuning on labeled

patches.

Thinking before scoring

We rated every Mini patch again with Gemma's thinking switched on (Table 13). Its AUROC on its own patches does not move. It also gives 100 to the same share of its own failing patches: 48 of 79 with thinking off and 49 with it on, 40 of them in both. Thinking makes its other scores more extreme. The own failing patches it does not score 100 drop from a mean of 34 to 14 (−19.7 [−34.0, −4.5]). Its mean leniency on its own failing patches therefore falls from +12.9 to +6.2 points. On other writers' failing patches it rises from −8.1 to −5.5. The self-judging gap shrinks because thinking makes Gemma less accurate on other writers' patches.

Table 13. Gemma 4 31B with reasoning off and with thinking on, Mini issues, mixed-harness patches: the 331 patches that all nine judges scored and Gemma scored in both modes (two patches have no thinking score). Leniency is own minus other writers' failing patches. `derived/thinking_mode_2026-10-22.json`.

	Reasoning off	Thinking on
AUROC, own patches	0.638	0.638
AUROC, other writers' patches	0.836	0.795
Own failing patches scored 100	61%	62%
Own failing patches scored 95 or above	66%	66%
Own failing patches below 100, mean score	34	14
Other writers' failing patches scored 95 or above	22%	35%
Leniency [95%]	+21.0 [11.5, 30.3]	+11.7 [1.1, 22.3]
Self-judging gap [95%]	+0.138 [0.063, 0.214]	+0.097 [0.010, 0.192]

Gemma's own reproduction scripts

While fixing a held-out issue, Gemma often writes a script that reproduces the bug and runs it again after its edit. Among Gemma's failing held-out runs, 79 added such a script to the patch and 73 ran it. In 63 of those 73, the last run of the script ended without an error. So in most failing runs Gemma had tested its patch with its own script, and the script passed. We counted the runs from the logs of Gemma's agent runs (`changelog/2026-11-03.md`, `spec/own-script-gate.md`).

A shell with the patch applied

Setup. We fixed the design and a stop rule in a written plan before any run (`spec/judge-as-actor.md`, exploratory). The judge was Gemma 4 31B, the 4-bit competition checkpoint served by vLLM on one NVIDIA A100 at temperature 0 with thinking off. It judged each of the 133 held-out patches it had scored 100 as a reader (52 failing, 81 passing). For each patch we built a sandbox from the issue's SWE-bench image, with the repository's environment active, and applied the patch without committing it. Gemma then ran in mini-SWE-agent 1.7.0 with a 15-step limit and was not told who wrote the patch. Its task was the issue, the patch, and this instruction (`scripts/judge_actor.yaml`):

```
This patch has been applied to the repository in /testbed.  Decide whether it resolves the issue.  First
write a test that fails on the original code and passes with the patch applied; git stash and git stash
pop switch between the two.  Then try at least one case the issue implies but does not show.  Use the
shell; do not decide from reading alone.  When done, end your answer with two lines:  TEST: <one shell
command that runs your test> and SCORE: <0 to 100, the chance the patch passes the maintainers' hidden
tests>.
```

After the run, the harness, not Gemma, ran Gemma's `TEST` command twice in the same sandbox: once with the patch reverted and once with it applied. A test run passes when it exits with code 0 and prints no traceback and no assertion error. A first pilot ran without the repository's environment active, so Gemma could not import the code. We discarded it and ran the pilot again with the environment fixed.

Stop rule. The full run, with other writers' patches, would start only if the shell lowered Gemma's approval enough. The plan stopped it if Gemma still gave 100 to more than half of the failing patches and to more than 80% of the passing ones.

Result. Gemma gave a score in 132 of 133 runs. It still gives 100 to 41 of the 51 failing patches (80% [69, 90]) and to 77 of the 81 passing patches (95% [90, 99]). The stop rule fired, so the full run did not start. In 31 of the 51 runs on failing patches (61%), Gemma's test fails without the patch and passes with it. Its test checks the behavior Gemma believes is right. The shell lowers Gemma's score on about one in five of these failing patches, but also on one in twenty of the passing ones. So it separates them only slightly, with an AUROC of 0.58 [0.52, 0.64] on the 132 patches. A run took 15 model calls on average and 140 seconds at the median (`derived/judge_as_actor_2026-11-04.json`).

LoRA fine-tuning on labeled patches

Setup. We fixed the design and its reading in a written plan before training (`spec/sft-pilot.md`, with addendum 1, exploratory). The data are all labeled judging prompts of the held-out issues and SWE-bench Lite: 2,488 patches on 463 issues, by Gemma 4 31B and nine other writers. We split them by issue, never by patch: 40% of the issues for training, 20% for development and 40% for evaluation. The training set has 990 patches on 184 issues (450 passing, 540 failing, 190 of them Gemma's own). The evaluation set has 979 patches on 185 issues, with 528 failing.

Training. Gemma's score is the logit of the word PASS minus the logit of FAIL after the paper's prompt. We trained a rank-16 LoRA adapter on the deployed checkpoint, expanded to bf16, with binary cross-entropy on that margin, two epochs, learning rate 2×10^{-5} and one seed. Training took 3.1 A100-hours. Each condition, adapter on and off, accepts a patch when its score clears a threshold set on the development issues to keep 90% of their passing patches.

Reading fixed in the plan. Fine-tuning improves checking if three things hold on the evaluation issues. False acceptance falls by at least 10 percentage points with an interval below zero, the adapter keeps at least 85% of the passing patches, and same-issue pair accuracy rises with an interval above zero. If the intervals of both false acceptance and pair accuracy include zero, the reading is "no measurable improvement". Anything else is inconclusive.

Result. The adapter does not reduce false acceptance by a measurable amount (Table 14). It accepts 48.9% of evaluation failing patches against 53.0% without it (−4.2 percentage points [−8.3, 0.0]), and it keeps fewer passing patches. AUROC and same-issue pair accuracy do not change. Among Gemma's own failing evaluation patches that it had scored 100 as a reader, from both data sets, the adapter still accepts 48 of 51, against 50 without it. The reading is "no measurable improvement". One seed, one learning rate and 184 training issues do not show that training cannot work.

Table 14. LoRA fine-tuning, evaluation issues (979 patches on 185 issues), adapter off and on, at the threshold set on the development issues. Intervals from 2,000 paired issue resamples. `derived/sft_pilot_run2_2026-11-05.json`.

Measure	Adapter off	Adapter on	On minus off [95%]
Failing patches accepted (528)	53.0%	48.9%	−4.2 [−8.3, 0.0]
Passing patches kept (451)	90.2%	88.5%	−1.7
AUROC	0.779	0.776	−0.002 [−0.021, 0.015]
Same-issue pair accuracy (658 pairs)	0.766	0.758	−0.008 [−0.032, 0.015]
Gemma's own failing patches it scored 100, accepted	50 of 51	48 of 51	−3.9 [−11.5, 3.8]

F Tests that Gemma did not write

This section backs Section 3.4 of the paper. The repository's existing tests encode what the project's developers expected. An agent can run them before and after a patch without a hidden test or a second model, hold back a patch that breaks one, and show Gemma what broke.

The existing-test check

Rule. We fixed the check and its reading in a written plan before running it (`spec/existing-test-gate.md`, `scripts/existing_test_gate.py`, exploratory). The check reads the patch and the repository only. It never reads the hidden tests, the reference patch or any label.

1. **Changed code.** The Python source files the patch modifies, and in them the innermost functions and classes around each changed line.
2. **Tests.** The repository's test files, ranked by whether their name matches a changed module and by how many changed functions or classes they name. The check keeps up to four, and a patch with no matching test file has no check.
3. **Runs.** In the issue's SWE-bench sandbox, with the repository's test command, each selected file runs twice on the unpatched code. A test that passes both times is stable. The selected files then run with the patch applied, and again if a stable test fails.
4. **Decision.** The check holds back the patch if a stable test fails, or disappears, in both runs with the patch. A patch with no stable test, or whose runs time out, is not held back.

We ran it on all 957 held-out patches and on the reference patch of each of the 193 issues. It checked 161 of Gemma's 180 patches and 184 of the 193 reference patches. Over all 1,150 patches, 59 had no matching test, 26 had a run that timed out or did not run, 8 had no stable test and 2 did not apply.

Reading fixed in the plan. The check would count as a fix for an agent under three conditions. It holds back at least 11 of the 52 failing patches Gemma scored 100, at most 4 of the 81 passing ones, and at most 5% of the reference patches.

Result. The check holds back 15 of the 52 failing patches Gemma scored 100, and 26 of all 85 of Gemma's failing patches (Table 15). It also holds back 7 of the 81 passing patches Gemma scored 100 and 18 of the 193 reference patches. By the plan, the check alone holds back too many correct patches to count as a fix. Section 3.4 of the paper therefore uses it only to choose which patches get a repair run.

Table 15. Patches the existing-test check holds back, held-out issues. A patch without a check counts as not held back. `derived/existing_test_gate_2026-11-05.json`.

Patches	Held back	Share [95%]
Gemma's failing patches it scored 100	15 of 52	29% [17, 42]
Gemma's failing patches, all	26 of 85	31% [21, 41]
Gemma's passing patches it scored 100	7 of 81	9% [3, 15]
Gemma's passing patches, all	9 of 95	9% [4, 16]
Reference patches	18 of 193	9% [5, 14]
Other writers' failing patches Gemma scored 100	34 of 136	25% [17, 34]
Other writers' passing patches	46 of 384	12% [7, 18]

Real regressions and old tests. After every decision was made, we ran each held-back patch's broken tests once more with the reference patch applied. A broken test that passes with the reference patch is a real regression. A broken test that also fails with the reference patch asserts the old behavior that the issue asks to change. For 14 of the 15 failing patches Gemma scored 100, the broken test passes with the reference patch, so these are real regressions that Gemma missed. For 5 of the 7 passing patches Gemma scored 100, the broken tests also fail with the reference patch. The 18 held-back reference patches are of this second kind by definition.

The repair run

Setup. We fixed the design and its reading in a written plan while the check was running, before any of its decisions were read (`spec/test-repair.md`, exploratory). Every one of Gemma's 180 patches that the check held back got one more Gemma run: 26 failing and 9 passing patches. Gemma 4 31B was served as for the shell judge (Section E). It ran in mini-SWE-agent 1.7.0 with its SWE-bench configuration, a 50-step limit, a 65,536-token context and the repository's environment active. It started from the issue's sandbox with the patch applied. Each patch ran in two conditions, with the same budget. The task was the issue text followed by one note:

- **Broken tests shown.** "A patch for this issue has already been applied to the repository (see `git diff`). Running the repository's existing tests shows that the patch breaks tests that passed before it:" Then, for each test file with a broken test, the command that runs it, the broken test names and the last 80 lines of its output. Then "Fix the patch so these tests pass again and the issue above stays solved. Do not edit the test files."
- **Second look.** "A patch for this issue has already been applied to the repository (see `git diff`). It may contain mistakes or break existing behaviour. Check it and fix any problem you find, keeping the issue above solved. Do not edit the test files."

We graded the submitted patch with the hidden tests. A run that did not submit keeps the original patch and its grade: 2 of 35 runs with the broken tests shown (model-call timeouts) and 1 of 35 second looks (the step limit).

Reading fixed in the plan. The broken tests help repair if more held-back failing patches pass with the broken tests shown than after a second look. The test is a one-sided exact sign test on the patches where the two conditions differ, with $p < 0.05$.

Result. With the broken tests shown, Gemma repairs 14 of the 26 held-back failing patches, against 6 after a second look (Table 16). Eight were repaired only with the tests shown and none only by the second look

($p = 0.004$), which meets the reading fixed before the run. Nine of the 14 are among the 52 failing patches Gemma scored 100. The same step breaks 4 of the 9 held-back passing patches, all of them patches Gemma scored 100. The second look breaks one, which Gemma had not scored 100. The sign test covers only the repair of held-back failing patches. Over all 35 held-back patches, the run with the tests shown does better than the second look on 9 and worse on 4 ($p = 0.13$, exploratory). Among the 133 patches Gemma scored 100, it does better on 6 and worse on 4 ($p = 0.38$).

Table 16. One repair run per held-back patch, held-out issues, graded with the hidden tests. `derived/test_repair_2026-11-05.json`, `derived/repair_trace_audit_2026-11-05.json`.

	Broken tests shown	Second look
Held-back failing patches repaired (of 26)	14	6
of them scored 100 by Gemma (of 52)	9	3
Held-back passing patches broken (of 9)	4	1
of them scored 100 by Gemma (of 81)	4	0
Passing among the 133 Gemma scored 100 (81 before)	86	84
Passing patches over all 180, after (95 before)	105	100

Why repairs fail and why correct patches break. We read every repair run that ended failing and every broken correct patch (`notes/repair-trace-audit-2026-11-05.md`, `scripts/repair_trace_audit_2026_11_05.py`). Of the 12 held-back failing patches that stay failing with the tests shown, 11 were submitted by Gemma after 7 to 38 model calls, with steps left, and one ended in a model-call timeout. So the runs did not fail for lack of steps.

The most common failure is a conflict between the issue and an old test. The note asks Gemma to make the broken tests pass and keep the issue solved, and when an old test asserts the behavior the issue changes, Gemma often restores the old behavior. In 3 of the 4 correct patches it broke, the broken test also fails with the reference patch, and Gemma restored the old behavior to make it pass. The fourth, django-11276, was broken by an editing error. Other failed repairs come from editing errors and from edits Gemma did not check. On sympy issue 12481 Gemma wrote that it had "re-introduced the disjointness check to make `test_args` pass, although this contradicts the PR description".

Breaking the tie at 100 by fewer model calls

Section 3.4 of the paper closes with a way to set Gemma's verdict aside without repairing anything. We fixed three tie-breaks on the Mini issues before computing any held-out value (Table 17; `spec/tie-break.md`). Inside Gemma's score, each ranks one kind of patch higher: fewer concern words in Gemma's answer, a shorter answer, or fewer model calls in Gemma's agent run. Only our own runs record model calls, so that tie-break was chosen on the 32 matched-harness Mini patches Gemma scored 100. On the held-out issues, concern words do not help, a shorter answer helps a little, and fewer model calls help most.

Ranking Gemma's 180 held-out patches by its score and then by fewer model calls raises the share of the top quarter that passes from 61% to 78%. The rule "Gemma scores 100 and the run used at most 9 model calls" submits 41 held-out patches (22.8%). Of these, 78.0% pass. Inside the tie, the number of model calls correlates with the outside judges' mean score (Spearman −0.57), so run length may partly stand for how hard the issue is.

Table 17. Tie-breaks inside Gemma's score, fixed on the Mini issues before the held-out test. AUROC inside the tie separates passing from failing patches among those Gemma scored 100. Gains are share passing minus Gemma's, in points, with 95% intervals. `derived/tie_break_2026-10-22.json`.

Tie-break	AUROC in tie, Mini	AUROC in tie, held-out	Top quarter	Top half
Fewer concern words	0.527 (142)	0.533 [0.438, 0.630]	+6.1 [−4.2, 16.8]	0.0 [−5.0, 5.5]
Shorter answer	0.621 (142)	0.626 [0.529, 0.717]	+10.9 [0.3, 21.7]	+4.1 [−1.4, 10.5]
Fewer model calls	0.731 (32)	0.679 [0.578, 0.769]	+16.9 [5.7, 26.8]	+10.8 [4.2, 16.5]

G Cost

Table 18. API cost as reported by OpenRouter per call. gpt-oss-120b calls were reported at zero cost by their provider. Gemma's 250 agent runs took 49 agent-hours on one NVIDIA L40S of a university cluster, at no charge. Gemma's LiveCodeBench programs were written on one NVIDIA A100 of our lab, at no charge.

Run	Calls	Cost (USD)
Four judges on Gemma's own attempts (ratings reused)	3,244	0.53
Nine judges, Mini issues, mixed-harness patches	3,292	2.66
Gemma 4 31B with thinking on, Mini issues	401	0.51
Nine judges, held-out and matched-harness patches	9,857	8.66
Seven judges, SWE-bench Lite (Section C)	10,526	2.44
Six judges, LiveCodeBench, with an 8-call cost test	5,319	0.98
Total	32,639	15.78